

why not-so-popular and domain-specific languages matter, then look at what parameters the language of the year ought to be judged by, and then analyse data to decide which language gets the award. We'll also look at relatively new languages and check the general trends in the language space to guess how languages might look after a few years. There's a lot to cover (and interesting aspects, too) in these topics, so this article does get a bit lengthy—bear with me.

Why programming languages matter

I don't know why I like programming languages, but I just do. When I discuss languages with others, I find there are many like me with a similar interest. I think 'interested' is an understatement—people are passionate, affectionate or sometimes even emotional about languages! Why does this happen?

I think programming languages have a special place in a programmer's life unlike 'technologies' or gizmos. As programmers, we use languages on a day-to-day basis and it is the primary means through which we get our work done. When we create scripts, we use scripting languages (like Perl). When we put data in an organised format, we often use markup languages (like XML). We interact with a database using SQL (a data manipulation language). But we usually don't recognise languages in other forms. For example, when I say 'awk' is a programming language, many are surprised—lots of people think it is just a 'tool'!

Programming is the creative activity that separates computer science from other engineering disciplines. So, it's not surprising that 50 per cent of the winners of the Turing Award—the highest in computer science [http://en.wikipedia.org/wiki/Turing_Award], a.k.a. the 'Nobel prize of computer science', are either inventors or very closely associated with programming/programming languages! For example, look at the following list—Dijkstra & Peter Naur (Algol), Backus (Fortran), Iverson (APL), Ritchie (C), Dahl & Nygaard (Simula) and Alan Kay (Smalltalk). It is clear that algorithms, data-structures, programming and programming languages matter.

Languages galore!

One of my favourite pictures is the language evolution tree [<http://www.digibarn.com/collections/posters/tongues/tongues.jpg>]. I know almost nothing about most of the languages listed in the tree. However, I have made an honest attempt to learn or read more about them.

I tried learning or reading about Smalltalk: I liked the erstwhile *Smalltalk Report* magazine and its numerous discussions on patterns; I also liked *The Little Smalltalk* book by Tim Budd, and while I was trying to read its few thousand lines of source code I was surprised to find how small the implementation of the interpreter is and how tiny—just a few lines of code—its GC implementation was!

Sometime back I tried reading about Oberon: I was just curious to know how Niklaus Wirth (creator of Oberon

and Pascal) was trying to come up with a better language than Pascal, but was put off by its ugly syntax—the mix of uppercase and lowercase letters, for example. I came to know about the Apple-backed language, Dylan, once touted as a competitor to Java, but got lost in the innumerable details even in the beginners tutorial book. I read a book on Objective-C and got curious when I learned about the language wars between C++ and Objective-C in the 80s, a war that C++ won. I also got interested in how features like the 'interface' from Java was directly influenced by Objective-C's interface feature. I discovered ActionScript (awkward name for a language—are other scripting languages inactive?), and I recently read an article in artima.com on Java vs AS3...

These lesser-known languages do matter a lot and we'll never know which might eventually get more popular. For example, Ruby, Python, Lua and ActionScript were relatively unknown languages just a few years back, and now they are more widely known and used. New languages are born by getting features from well-known as well as lesser-known languages. For example, Java's languages features are inspired by Mesa, Modula-3, Eiffel, Pascal (UCSD Pascal, Object Pascal and Delphi), Objective-C, Smalltalk, Ada and C++.

Apart from the technical strength of the language, other aspects like market forces, the company sponsorship, etc, play an important role in making a language popular. For example, the language landscape would be different today if Dylan was more successful than Java—both were released around the same time in 1995—one was backed by Apple and another by Sun.

Selecting the most suitable language

Sometimes, I am surprised to find many people working in languages that I haven't even heard of: M and Poplog, for example. Why do people use relatively unknown, uncommon, rare or even obscure languages? We'll look at an analogy to understand that.

When I came to Bangalore, I liked spotting the cute little Reva cars. Reva is special because it is a fully-electric, city car, like an autorickshaw-sized two seater. It is not really cheap (the basic model costs approx Rs 4 lakhs); but running costs are low, and it doesn't pollute. A housewife wanting to drive the car for short distances and with her own house (that makes charging the car easy) is likely to buy the car. In other words, it has a unique value proposition, and if the value that the car provides satisfies your need, you will buy a relatively uncommon car. Selecting a programming language is akin to this.

We need to look out for languages that are most effective in getting our work done. It might be surprising to many of you, but most effective languages are not general-purpose languages: Domain-specific languages (DSLs) are often very good for getting our work done. Recently, I discovered that I can use languages for doing common tasks like document preparation and creating diagrams(!).